

Project Documentation

Project Name: RBAC Project and Task Management System

Prepared by: Kartik Agasimani

This document gives a human-readable explanation of the project, the decisions taken, the implemented modules, the current limitations, and the planned improvements.

Objective

The objective of this project is to provide a role-based task management system where an administrator can manage team members, create tasks, assign work, track task status, and monitor productivity through a dashboard. The project is aligned with the practical assignment requirement for a role-based project and task management system.

Application Flow

- The admin logs in using secure credentials.
- The admin can add team members such as Project Managers and Employees.
- The admin creates tasks with title, priority, date, status, and assigned users.
- The dashboard displays live statistics based on active tasks.
- Tasks can be viewed in board and list views.
- Tasks can be moved to trash and reviewed from the trash page.
- Docker support is available for containerized local setup.

Architecture

The project follows a client-server architecture. The React frontend handles user interaction and page rendering. The Express backend exposes REST APIs for authentication, users, tasks, and dashboard statistics. MongoDB Atlas stores the application data. JWT cookies are used for maintaining authenticated sessions.

Component	Responsibility
React frontend	Displays dashboards, forms, task views, team pages, and trash pages.
Express backend	Handles APIs, authentication, validation, and database operations.
MongoDB Atlas	Stores users, tasks, notifications, and related records.
JWT cookie	Keeps the logged-in user authenticated securely.
Docker Compose	Runs frontend and backend together in containers.

Implemented Modules

- Authentication: Admin login with JWT and HTTP-only cookies.
- User Management: Add, view, activate, deactivate, and delete team members.
- Task Management: Create, assign, view, and categorize tasks by status.

- Dashboard: Live task count, completed tasks, in-progress tasks, todo tasks, recent tasks, and active users.
- Trash Management: Deleted tasks are moved to trash and can be reviewed.
- API Documentation: Key endpoints are documented for testing and review.
- Docker Configuration: Dockerfiles and docker-compose.yml are included.

How the Project Matches the Assignment

Assignment Requirement	Current Project Status
Authentication and RBAC	Implemented for admin and role foundation.
Role-based dashboards	Admin dashboard implemented with live task data.
Task management	Implemented with task creation, status, priority, date, and assigned users.
Team management	Implemented for users and roles.
Activity and notification foundation	Notification model exists and can be extended.
Reports and analytics	Basic dashboard analytics implemented.
Docker support	Docker configuration added and tested.
API documentation	Separate API documentation created.

Known Limitations

- A full project management module is not implemented yet.
- Project Manager and Employee-specific dashboards are planned but not fully separated.
- Work logs and manager replies are not fully implemented in this version.
- Email deadline reminders and overdue alerts are planned enhancements.
- Audit logs are planned for future implementation.
- The assignment mentions MySQL, but MongoDB Atlas was used to complete a working prototype quickly.

Future Scope

- Add project creation with assigned Project Managers.
- Restrict Project Managers to assigned projects only.
- Allow Employees to view and update only assigned tasks.
- Add work log submission with hours and attachments.
- Add manager replies to work logs.
- Add email reminders using Node Cron and Nodemailer.
- Add audit log collection for every important action.
- Create a normalized MySQL version of the schema.
- Add Swagger documentation and automated tests.

Demo Credentials

Email: admin@test.com

Password: 123456

Conclusion

This project delivers a working core version of the role-based task management workflow. It includes authentication, team management, task management, live dashboard statistics, trash handling, API documentation, and Docker configuration. The system is structured so that the remaining modules can be added cleanly in future iterations.